

src\case_studies\E_blockbeam\pd_controller.py

Generated: 2026-02-20 21:11:27

pd_controller.py

src\case_studies\E_blockbeam\pd_controller.py

```
# 3rd-party
import numpy as np

# Local (controlbook)
from . import params as P
from .. import common
from ..control.pd import PD
from ..control import utils_design

class BlockbeamControllerPD(common.ControllerBase):
    def __init__(self):
        # tuning parameters
        tr_theta = 1.0 # just bad all around
        tr_theta = 0.3 # better starting point, maybe even 0.4
        tr_theta = 0.18 # tuned to barely saturate for a single step
        zeta_theta = 0.707
        M = 10 # time separation factor between inner and outer loop
        tr_z = tr_theta * M
        zeta_z = 0.707

        # system parameters
        a0_inner = P.tf_inner_den[-1]
        b0_inner = P.tf_inner_num[-1]
        b0_outer = P.tf_outer_num[-1]

        # Inner loop
        wn_theta = utils_design.get_natural_frequency(tr_theta, zeta_theta)
        kp_theta = wn_theta**2 / b0_inner
        kd_theta = 2 * zeta_theta * wn_theta / b0_inner
        # kp_theta, kd_theta = utils_design.get_pd_gains_from_des_CE(
        #     wn_theta, zeta_theta, inner_b0
        # )
        self.theta_pd = PD(kp_theta, kd_theta)
        print(f"Inner loop (theta): {kp_theta = :.3f}, {kd_theta = :.3f}")

        # DC gain of inner loop
        DC_gain = (b0_inner * kp_theta) / (a0_inner + b0_inner * kp_theta)
        print(f"{DC_gain = }") # a0 is 0, so DC_gain is 1

        # Outer loop
        effective_b0_outer = b0_outer * DC_gain
        wn_z = utils_design.get_natural_frequency(tr_z, zeta_z)
        kp_z = wn_z**2 / effective_b0_outer
        kd_z = 2 * zeta_z * wn_z / effective_b0_outer
        self.z_pd = PD(kp_z, kd_z)
        print(f"Outer loop (z): {kp_z = :.3f}, {kd_z = :.3f}")

    def update_with_state(self, r, x):
        z_ref = r[0]
        z, theta, zdot, thetadot = x

        # outer loop control
        theta_ref = self.z_pd.update_modified(z_ref, z, zdot)
        r[1] = theta_ref # if you want to visualize the "reference" angle

        # inner loop control
        F_tilde = self.theta_pd.update_modified(theta_ref, theta, thetadot)
        F_fl = P.m1 * P.g * (z / P.length) + P.m2 * P.g / 2.0
```

```
u_unsat = np.array([F_tilde + F_f1])  
u = self.saturate(u_unsat, u_max=P.force_max)  
return u
```